

Rapport d'activité – Rendu 3

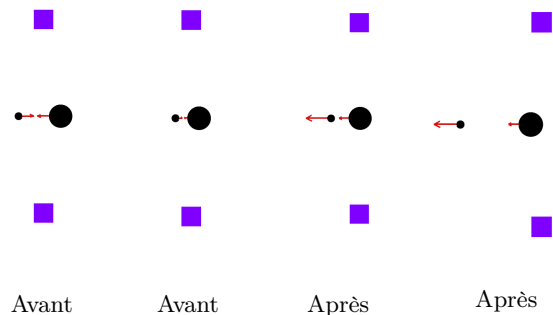
Partie 1 — Dynamiques du jeu

1.1 Changements structurels par rapport au Rendu 2 (optionnel)

- **Ball** : ajout des méthodes `translate()` et `setDelta()` pour rendre la classe mutable.
- **Paddle** : ajout de l'attribut `last_delta` et de la méthode `getDelta()` pour la collision balle-raquette.
- **Brick** : ajout de la méthode virtuelle pure `hit()` dans la hiérarchie de classes.
- **Game** : réécriture complète de `update()` avec les trois phases de la dynamique et toutes ses sous-fonctions.

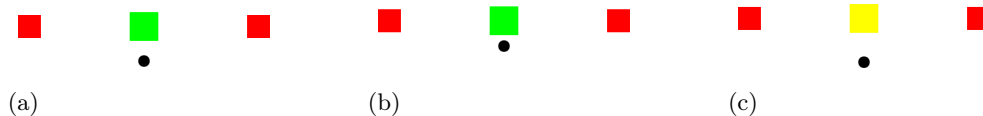
1.2 Collision de deux balles de rayons différents

Dans `update()`, `detect_collisions()` parcourt toutes les paires et signale un contact dès que la distance entre centres est $< r_a + r_b + \epsilon$. `bounce_ball()` projette alors le delta de chaque balle sur l'axe des centres (produit scalaire `dot`), calcule les facteurs d'échange $f_i = r_b / (r_a + r_b)$ et $f_j = r_a / (r_a + r_b)$, puis met à jour les deltas via `setDelta()` : la composante normale de A augmente, celle de B diminue. Selon l'intensité, B peut inverser sa direction ou simplement ralentir. Les composantes tangentielles restent inchangées. Les balles sont ensuite séparées (`translate()`) et leurs deltas bornés à `delta_norm_max`.

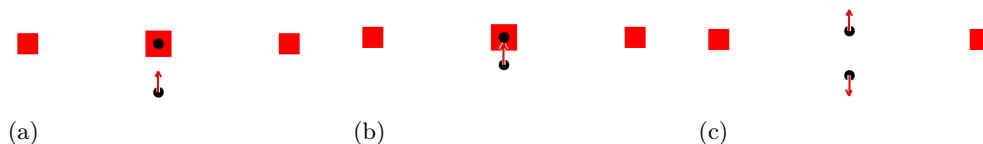


1.3 Destruction de chaque type de brique

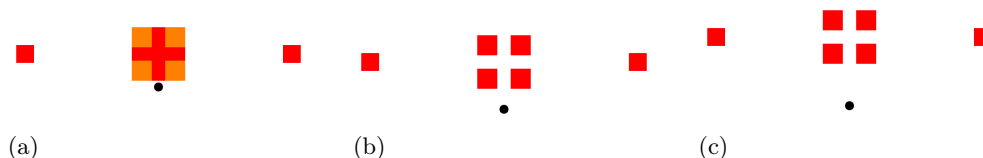
RainbowBrick : `bounce_brick()` détermine la face touchée en comparant les recouvrements horizontal et vertical, réfléchit le delta de la balle via `setDelta()`, puis appelle `hit()` : le compteur `hit_points` est décrémenté et l'index de couleur est incrémenté modulo la taille du cycle. Quand `hit_points` atteint zéro, `hit()` renvoie `true` et la brique est supprimée : (a) état initial, (b) après un impact, (c) détruite.



BallBrick : (a) brique intacte. (b) `bounce_brick()` sauvegarde `old_delta = ball.getDelta()` avant toute réflexion, réfléchit la balle frappante, puis appelle `hit()` : la brique est immédiatement détruite (`hit()` renvoie `true`) et un **Ball** centré sur la brique est poussé dans `new_balls` avec `old_delta`, capturant ainsi la direction avant rebond. (c) la balle frappante continue avec son delta réfléchi ; la nouvelle balle part dans la direction `old_delta`, c'est-à-dire celle d'avant le rebond. Les deux sont intégrées au vecteur `balls` à l'itération suivante.



SplitBrick : (a) brique intacte. (b) `bounce_brick()` appelle `hit()` sur la brique mère : la taille fille $s' = (\text{size} - \text{split_brick_gap})/2$ est calculée. Si $s' \geq \text{brick_size_min}$, quatre nouvelles **SplitBrick** sontinstanciées aux quatre quadrants et ajoutées via `bricks.push_back()`. (c) les filles intègrent la liste active et peuvent à leur tour se diviser ; si $s' < \text{brick_size_min}$, elles sont simplement détruites à l'impact sans spawner.



Partie 2 — Travail en groupe et bilan

2.1 Activité individuelle

- **Liam Van Camp** : Restructuration complète de `game::update()` en sous-fonctions (`move_balls`, `move_ball`, `move_ball_paddle`, `move_balls_paddle`). Implémentation de la physique balle-raquette (`bounce_paddle`, dépénétration) et balle-balle (`bounce_ball`, `detect_collisions`). Mise à jour de `game::update()` avec les trois phases et les conditions de fin de partie. Mise à jour de `Paddle` avec `last_delta` et `get_delta()`.
- **Victor Henri Willy Eder** : Mise à jour de `Ball` avec `translate()` et `set_delta()`. Ajout de la méthode virtuelle pure `hit()` dans la hiérarchie `Brick` avec implémentation pour `RainbowBrick` (décrément `hit_points`), `BallBrick` (génération d'une nouvelle balle) et `SplitBrick` (division en 4 sous-briques). Vérification des conventions de programmation sur l'ensemble du code. Mise à jour de `gui` et `game` pour l'affichage des messages de fin de partie.

2.2 Méthodologie

Organisation et outils : Même méthode que les rendus précédents, avec Git partagé et communication via Discord et WhatsApp. Le Rendu 3 a nécessité plus de coordination que les rendus précédents car les modules étaient fortement liés notamment `Ball`, `Brick`, `Paddle` et `Game` qui devaient évoluer simultanément.

Ordre de développement et tests : Nous avons commencé par définir la structure globale de `game::update()` avant d'en implémenter le détail. Nous avons ensuite rendu `Ball` et `Paddle` mutables, puis ajouté `hit()` aux briques, avant de compléter `game::update()`. Chaque étape a été testée visuellement dans l'interface graphique, notamment grâce au mode pas-à-pas (bouton *Step*) qui permet d'avancer image par image et d'observer précisément le comportement des entités. De plus des fichiers test spécifiques ont été créés pour valider les nouvelles fonctionnalités une fois implémentées.

Travail simultané vs indépendant : Une moitié du travail a été réalisée de manière simultanée et coordonnée, l'autre moitié de manière indépendante. Avec le recul, cette proportion était bien adaptée au Rendu 3 car les dépendances entre modules nécessitaient une communication constante, contrairement aux Rendus 1 et 2 où les modules étaient plus indépendants.

2.3 Bug le plus fréquent

Le bug le plus fréquent provenait d'un mauvais ordre d'appel des fonctions dans `update()` : déplacer une balle avant ou après le calcul de collision changeait complètement le résultat. Le cas le plus récurrent concernait le rebond sur la raquette : la balle la traversait ou rebondissait dans la mauvaise direction car le déplacement était appliqué avant la dépénétration, plaçant la balle de l'autre côté au moment du calcul. La correction a consisté à toujours effectuer la dépénétration avant la mise à jour de la position, et à s'assurer que `get_delta()` de la raquette était appelé au bon moment.

2.4 Usage d'un outil d'IA

Nous avons utilisé Claude (Anthropic) tout au long du projet, principalement pour contrôler le code écrit et corriger les erreurs plus rapidement. Cette approche s'est avérée efficace, mais nécessitait une vérification systématique : faute de vision globale du projet, l'outil introduisait parfois de nouvelles erreurs ailleurs en corrigeant un point précis. Il a également été utilisé pour implémenter certaines parties spécifiques, comme les calculs d'impulsion entre balles ou la lecture de fichiers selon les conventions vues en série d'exercices. Dans ces cas, il était impératif de fournir une structure claire en amont. Enfin, l'IA a été utilisée pour la mise en forme et la formulation des rapports. En résumé, le gain de temps était clairement visible pour la détection et la correction d'erreurs. Néanmoins le temps gagné à la génération était en partie compensé par le travail de structuration préalable indispensable.

2.5 Conclusion

Code : Certains comportements propres à `Ball` ou `Brick` sont gérés depuis `Game` alors qu'ils auraient pu résider directement dans ces classes. Nous le faisons déjà en partie `hit()` dans `Brick`, `translate()` et `set_delta()` dans `Ball` mais la démarche aurait pu être poussée plus loin pour alléger `Game`. En revanche, la séparation de `update()` en sous-fonctions bien délimitées (`bounce_brick`, `bounce_ball`, `detect_collisions`, etc.) s'est avérée un vrai atout : le code est lisible, chaque comportement est isolé et les modifications restent locales.

Environnement de travail : GitHub a été très utile, même s'il a fallu un temps d'adaptation initial. De plus, certaines séries d'exercices ont beaucoup aidé, mais une proportion trop grande de code y était détaillée par rapport à ce qui était couvert en cours. La structure de modularisation imposée était difficile à visualiser au départ, mais elle s'est révélée indispensable et a grandement facilité le travail par la suite.